

# Raft (algorithm)

**Raft** is a consensus algorithm designed as an alternative to the Paxos family of algorithms. It was meant to be more understandable than Paxos by means of separation of logic, but it is also formally proven safe and offers some additional features.<sup>[1]</sup> Raft offers a generic way to distribute a state machine across a cluster of computing systems, ensuring that each node in the cluster agrees upon the same series of state transitions. It has a number of open-source reference implementations, with full-specification implementations in Go, C++, Java, JavaScript, and Scala.<sup>[2]</sup> It is named after Reliable, Replicated, Redundant, And Fault-Tolerant.<sup>[3]</sup>

Raft is not Byzantine fault tolerant in the base form; the nodes trust the elected leader, and the algorithm assumes all participants are trustworthy, yielding only crash fault tolerance. However there exist variants that use cryptographic signatures to achieve Byzantine fault tolerance.<sup>[4]</sup>

## Raft



The Raft consensus algorithm mascot.

**Class** Consensus algorithm

## Basics

Raft achieves consensus via an elected leader. A server in a raft cluster is either a *leader* or a *follower*, and can be a *candidate* in the precise case of an election (leader unavailable). The leader is responsible for log replication to the followers. It regularly informs the followers of its existence by sending a heartbeat message. Each follower has a timeout (typically between 150 and 300 ms) in which it expects the heartbeat from the leader. The timeout is reset on receiving the heartbeat. If no heartbeat is received the follower changes its status to candidate and starts a leader election.<sup>[1][5]</sup>

## Approach of the consensus problem in Raft

Raft implements consensus by a leader approach. The cluster has one and only one elected leader which is fully responsible for managing log replication on the other servers of the cluster. It means that the leader can decide on new entries' placement and establishment of data flow between it and the other servers without consulting other servers. A leader leads until it fails or disconnects, in which case surviving servers elect a new leader.

The consensus problem is decomposed in Raft into two relatively independent subproblems listed down below.

## Leader election

When the existing leader fails or when the algorithm initializes, a new leader needs to be elected.

In this case, a new *term* starts in the cluster. A term is an arbitrary period of time on the server for which a new leader needs to be elected. Each term starts with a leader election. If the election is completed successfully (i.e. a single leader is elected) the term keeps going with normal operations orchestrated by the new leader. If the election is a failure, a new term starts, with a new election.

A leader election is started by a *candidate* server. A server becomes a candidate if it receives no communication by the leader over a period called the *election timeout*, so it assumes there is no acting leader anymore. It starts the election by increasing the term counter, voting for itself as new leader, and sending a message to all other servers requesting their vote. A server will vote only once per term, on a first-come-first-served basis. If a candidate receives a message from another server with a term number larger than the candidate's current term, then the candidate's election is defeated and the candidate changes into a follower and recognizes the leader as legitimate. If a candidate receives a majority of votes, then it becomes the new leader. If neither happens, e.g., because of a split vote, then a new term starts, and a new election begins.<sup>[1]</sup>

Raft uses a randomized election timeout to ensure that split vote problems are resolved quickly. This should reduce the chance of a split vote because servers won't become candidates at the same time: a single server will time out, win the election, then become leader and send heartbeat messages to other servers before any of the followers can become candidates.<sup>[1]</sup>

## Log replication

The leader is responsible for the log replication. It accepts client requests. Each client request consists of a command to be executed by the replicated state machines in the cluster. After being appended to the leader's log as a new entry, each of the requests is forwarded to the followers as AppendEntries messages. In case of unavailability of the followers, the leader retries AppendEntries messages indefinitely, until the log entry is eventually stored by all of the followers.

Once the leader receives confirmation from half or more of its followers that the entry has been replicated, the leader applies the entry to its local state machine, and the request is considered *committed*.<sup>[1][5]</sup> This event also commits all previous entries in the leader's log. Once a follower learns that a log entry is committed, it applies the entry to its local state machine. This ensures consistency of the logs between all the servers through the cluster, ensuring that the safety rule of Log Matching is respected.

In the case of a leader crash, the logs can be left inconsistent, with some logs from the old leader not being fully replicated through the cluster. The new leader will then handle inconsistency by forcing the followers to duplicate its own log. To do so, for each of its followers, the leader will compare its log with the log from the follower, find the last entry where they agree, then delete all the entries coming after this critical entry in the follower log and replace it with its own log entries. This mechanism will restore log consistency in a cluster subject to failures.

# Safety

## Safety rules in Raft

Raft guarantees each of these safety properties:

- **Election safety:** at most one leader can be elected in a given term.
- **Leader append-only:** a leader can only append new entries to its logs (it can neither overwrite nor delete entries).
- **Log matching:** if two logs contain an entry with the same index and term, then the logs are identical in all entries up through the given index.
- **Leader completeness:** if a log entry is committed in a given term then it will be present in the logs of the leaders since this term.
- **State machine safety:** if a server has applied a particular log entry to its state machine, then no other server may apply a different command for the same log.

The first four rules are guaranteed by the details of the algorithm described in the previous section. The State Machine Safety is guaranteed by a restriction on the election process.

## State machine safety

This rule is ensured by a simple restriction: a candidate can't win an election unless its log contains all committed entries. In order to be elected, a candidate has to contact a majority of the cluster, and given the rules for logs to be committed, it means that every committed entry is going to be present on at least one of the servers the candidates contact.

Raft determines which of two logs (carried by two distinct servers) is more up-to-date by comparing the index term of the last entries in the logs. If the logs have a last entry with different terms, then the log with the later term is more up-to-date. If the logs end with the same term, then whichever log is longer is more up-to-date.

In Raft, the request from a candidate to a voter includes information about the candidate's log. If its own log is more up-to-date than the candidate's log, the voter denies its vote to the candidate. This implementation ensures the State Machine Safety rule.

## Follower crashes

If a follower crashes, *AppendEntries* and *vote* requests sent by other servers will fail. Such failures are handled by the servers trying indefinitely to reach the downed follower. If the follower restarts, the pending requests will complete. If the request has already been taken into account before the failure, the restarted follower will just ignore it.

## Timing and availability

Timing is critical in Raft to elect and maintain a steady leader over time, in order to have a perfect availability of the cluster. Stability is ensured by respecting the *timing requirement* of the algorithm:

$$broadcastTime \ll electionTimeout \ll MTBF$$

- *broadcastTime* is the average time it takes a server to send a request to every server in the cluster and receive responses. It is relative to the infrastructure used.
- *MTBF (Mean Time Between Failures)* is the average time between failures for a server. It is also relative to the infrastructure.
- *electionTimeout* is the same as described in the Leader Election section. It is something the programmer must choose.

Typical numbers for these values can be 0.5 ms to 20 ms for *broadcastTime*, which implies that the programmer sets the *electionTimeout* somewhere between 10 ms and 500 ms. It can take several weeks or months between single server failures, which means the values are sufficient for a stable cluster.

## Cluster Membership Changes

To address cluster membership change issues that can arise in Raft, the algorithm introduces joint consensus, a transitional configuration phase. Joint consensus works as follows:<sup>[6]</sup>

Given server configuration  $C_{old}$ , the old server configuration, and  $C_{new}$ , the new configuration:<sup>[6]</sup>

- Log entries are committed to every server in  $C_{old}$  and  $C_{new}$ .<sup>[6]</sup>
- Any server from  $C_{old}$  and  $C_{new}$  can be leader.<sup>[6]</sup>
- Agreement for elections and log entry commits requires majorities from both  $C_{old}$  and  $C_{new}$ .<sup>[6]</sup>

Once the joint consensus is done (that is, the special  $C_{new}$  configuration entry is replicated to a majority of  $C_{new}$  servers' logs), the system fully transitions to the new configuration.<sup>[6]</sup>

However, there are three issues that arise with this new configuration, of which Raft addresses:<sup>[7]</sup>

1. New servers with no log entries. Raft introduces a phase before the configuration change where servers with no log entries are not considered part of the majority in elections, but they will have entries replicated to them. This happens until the server is fully caught up with entries.<sup>[7]</sup>
2. Cluster leader isn't in  $C_{new}$ . The cluster leader will stop being leader and return to follower state. Specifically, it will continue to replicate log entries, but will not count itself as majorities.<sup>[7]</sup>
3. Disruptions from servers in  $C_{old}$ . If a server believes that a current leader exists, any RequestVote RPCs (RPCs to gather votes for a leader election) will be disregarded.<sup>[7]</sup>

## Log Compaction

An extension to Raft is log compaction, where each server takes snapshots of its committed entries and saves it to stable storage, along with the index of its last entry and the term in the snapshotted log. The leader occasionally sends its snapshots to servers that are lagging behind in its log. When the server receives this snapshot, it will either discard its entire log if it's superseded by the snapshot, or only the entries up to the latest one in the snapshot.<sup>[8]</sup>

## Issues

While Raft aims to be an alternative of Paxos and more understandable than Paxos, several issues arise.

## Leader Bottleneck

Raft uses a single leader model, where client requests, reads and writes, and log replications goes through a single leader. This means that there is a single point of failure and a bottleneck in performance. Furthermore, it does not scale with increasing server workload.<sup>[9]</sup>

## Reconfiguration

Raft's membership change system has not been formally verified to be correct. This means that implementing it is very risky, as there can be many potential bugs and errors. Diego Ongaro, one of the co-authors, has tried to create a formal safety proof, but there are no plans to continue developing it due to how complicated it is.<sup>[10]</sup> In 2014, a safety bug was found relating to single server membership changes.<sup>[10]</sup>

## Byzantine Faults

Raft, like other consensus algorithms, ensures that there can never be an incorrect result under all non-Byzantine conditions.<sup>[11]</sup> This means that Raft is not a Byzantine fault tolerant algorithm. A 2023 study found that blockchain systems based on Raft are vulnerable to Byzantine attacks because of the lack of authentication on the client side.<sup>[12]</sup>

## Extensions

---

The dissertation “Consensus: Bridging Theory and Practice” by one of the co-authors of the original paper describes extensions to the original algorithm:<sup>[13]</sup>

- Pre-Vote: when a member rejoins the cluster, it can depending on timing trigger an election although there is already a leader. To avoid this, pre-vote will first check in with the other members. Avoiding the unnecessary election improves the availability of cluster, therefore this extension is usually present in production implementations.
- Leadership transfer: a leader that is shutting down orderly can explicitly transfer the leadership to another member. This can be faster than waiting for a timeout. Also, a leader can step down when another member would be a better leader, for example when that member is on a faster machine.

## Production use of Raft

---

- CockroachDB uses Raft in the Replication Layer.<sup>[14]</sup>
- Etcd uses Raft to manage a highly-available replicated log <sup>[15]</sup>
- Hazelcast uses Raft to provide its CP Subsystem, a strongly consistent layer for distributed data structures. <sup>[16]</sup>
- IBM MQ uses Raft to manage a highly-available replicated log. <sup>[17]</sup>
- MongoDB uses a variant of Raft in the replication set.
- Neo4j uses Raft to ensure consistency and safety. <sup>[18]</sup>
- RabbitMQ uses Raft to implement durable, replicated FIFO queues. <sup>[19]</sup>
- ScyllaDB uses Raft for metadata (schema and topology changes) <sup>[20]</sup>
- Splunk Enterprise uses Raft in a Search Head Cluster (SHC) <sup>[21]</sup>

- TiDB uses Raft with the storage engine TiKV.<sup>[22]</sup>
- YugabyteDB uses Raft in the DocDB Replication <sup>[23]</sup>
- ClickHouse uses Raft for in-house implementation of ZooKeeper-like service <sup>[24]</sup>
- Redpanda uses the Raft consensus algorithm for data replication <sup>[25]</sup>
- Apache Kafka Raft (KRaft) uses Raft for metadata management.<sup>[26]</sup>
- NATS Messaging uses the Raft consensus algorithm for Jetstream cluster management and data replication <sup>[27]</sup>
- Camunda uses the Raft consensus algorithm for data replication <sup>[28]</sup>

## References

---

1. Ongaro, Diego; Ousterhout, John (2013). "In Search of an Understandable Consensus Algorithm" (<https://raft.github.io/raft.pdf>) (PDF).
2. "Raft Consensus Algorithm" (<https://raft.github.io/>). 2014.
3. Why the "Raft" name? (<https://groups.google.com/forum/#!topic/raft-dev/95rZqptGpmU>)
4. Christopher Copeland and Hongxia Zhong (2016). Tangaroa: a Byzantine Fault Tolerant Raft ([https://www.scs.stanford.edu/14au-cs244b/labs/projects/copeland\\_zhong.pdf](https://www.scs.stanford.edu/14au-cs244b/labs/projects/copeland_zhong.pdf))
5. Ben B. Johnson. "Raft: Understandable Distributed Consensus" (<http://thesecretlivesofdata.com/raft/>). *The Secret Lives of Data website*. Retrieved August 4, 2021.
6. "In Search of an Understandable Consensus Algorithm (Extended Version)" (<https://raft.github.io/raft.pdf>) (PDF). *raft.github.io*. Retrieved 2025-10-01.
7. "In Search of an Understandable Consensus Algorithm (Extended Version)" (<https://raft.github.io/raft.pdf>) (PDF). *raft.github.io*. Retrieved 2025-10-01.
8. "In Search of an Understandable Consensus Algorithm (Extended Version)" (<https://raft.github.io/raft.pdf>) (PDF). *raft.github.io*. Retrieved 2025-10-01.
9. Deyerl, Christian; Distler, Tobias (2019-03-25). "In Search of a Scalable Raft-based Replication Architecture" (<https://doi.org/10.1145/3301419.3323968>). *Proceedings of the 6th Workshop on Principles and Practice of Consistency for Distributed Data*. PaPoC '19. New York, NY, USA: Association for Computing Machinery. pp. 1–7. doi:10.1145/3301419.3323968 (<https://doi.org/10.1145/3301419.3323968>). ISBN 978-1-4503-6276-4.
10. "bug in single-server membership changes" (<https://groups.google.com/g/raft-dev/c/t4xj6dJTP6E/m/d2D9LrWRza8J?pli=1>). *groups.google.com*. Retrieved 2025-10-06.
11. "In Search of an Understandable Consensus Algorithm (Extended Version)" (<https://raft.github.io/raft.pdf>) (PDF). *raft.github.io*. Retrieved 2025-10-01.
12. "Poster: Client-Side Byzantine Attacks on Raft Algorithm in Blockchain" ([https://www.ndss-symposium.org/wp-content/uploads/2023/02/NDSS2023Poster\\_paper\\_9949.pdf](https://www.ndss-symposium.org/wp-content/uploads/2023/02/NDSS2023Poster_paper_9949.pdf)) (PDF).
13. "CONSENSUS: BRIDGING THEORY AND PRACTICE" (<https://web.stanford.edu/~ouster/cgi-bin/papers/OngaroPhD.pdf>) (PDF).
14. "Replication Layer | CockroachDB Docs" (<https://www.cockroachlabs.com/docs/stable/architecture/replication-layer.html>). *www.cockroachlabs.com*. Retrieved 2022-06-21.
15. "Raft README" (<https://github.com/etcd-io/raft/blob/main/README.md>). *github.com*. Retrieved 2022-08-25.
16. "CP Subsystem" (<https://docs.hazelcast.com/impl/4.2/cp-subsystem/cp-subsystem>). *docs.hazelcast.com*. Retrieved 2022-12-24.
17. "Cloud native high availability with IBM MQ" (<https://community.ibm.com/community/user/blogs/david-ware1/2021/03/23/native-ha-cloud-native-high-availability>). *community.ibm.com*. 25 March 2021. Retrieved 2021-05-25.

18. "Leadership, routing and load balancing - Operations Manual" (<https://neo4j.com/docs/operations-manual/5/clustering/setup/routing/>). *Neo4j Graph Data Platform*. Retrieved 2022-11-30.
19. "Quorum Queues" (<https://www.rabbitmq.com/quorum-queues.html>). *RabbitMQ*. Retrieved 2022-12-14.
20. "ScyllaDB's Path to Strong Consistency: A New Milestone" (<https://www.scylladb.com/2023/05/04/scylladbs-path-to-strong-consistency-a-new-milestone/>). 4 May 2023.
21. "Handle Raft issues" (<https://docs.splunk.com/Documentation/Splunk/9.0.0/DistSearch/HandleRaftIssues>). *Splunk*. 2022-08-24. Retrieved 2022-08-24.
22. "Raft and High Availability" (<https://en.pingcap.com/blog/raft-and-high-availability/>). *PingCAP*. 2021-09-01. Retrieved 2022-06-21.
23. "Replication | YugabyteDB Docs" (<https://docs.yugabyte.com/preview/architecture/docdb-replication/replication/>). *www.yugabyte.com*. Retrieved 2022-08-19.
24. "ClickHouse Keeper" (<https://clickhouse.com/docs/en/guides/sre/keeper/clickhouse-keeper>). *clickhouse.com*. Retrieved 2023-04-26.
25. "Raft consensus algorithm" (<https://docs.redpanda.com/docs/get-started/architecture/#raft-consensus-algorithm>).
26. "KRaft Overview | Confluent Documentation" (<https://docs.confluent.io/platform/current/kafka-metadata/kraft.html>). *docs.confluent.io*. Retrieved 2024-04-13.
27. "JetStream Clustering" ([https://docs.nats.io/running-a-nats-service/configuration/clustering/jetstream\\_clustering](https://docs.nats.io/running-a-nats-service/configuration/clustering/jetstream_clustering)).
28. "Raft consensus and replication protocol" (<https://docs.camunda.io/docs/components/zeebe/technical-concepts/clustering/#raft-consensus-and-replication-protocol>).

## External links

---

- [Official website \(<https://raft.github.io/>\)](https://raft.github.io/) 
- 

Retrieved from "[https://en.wikipedia.org/w/index.php?title=Raft\\_\(algorithm\)&oldid=1369179570](https://en.wikipedia.org/w/index.php?title=Raft_(algorithm)&oldid=1369179570)"